Yuck! Four parameters might not seem that bad right now, but what happens when the requirements change and you need to collect other details? The method signature will keep growing. The model binder will bind the values quite happily, but it's not exactly clean code. Using the `[BindProperty]` approach doesn't really help either; you still have to clutter your `PageModel` with lots of properties and attributes!

Simplifying method parameters by binding to complex objects

A common pattern for any C# code when you have many method parameters is to extract a class that encapsulates the data the method requires. If extra parameters need to be added, you can add a new property to this class. This class becomes your binding model, and it might look something like the following listing.

Listing 16.4 A binding model for capturing a user's details

```
public class UserBindingModel
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Email { get; set; }
    public string PhoneNumber { get; set; }
}
```

NOTE In this book I primarily use `class` instead of `record` for my binding models, but you can use `record` if you prefer. I find the terseness that the `record` positional syntax provides is lost if you want to add attributes to properties, such as to add validation attributes, as you'll see in section 16.3. You can see the required syntax for positional property attributes in the documentation at http://mng.bz/Kex0.

With this model, you can update your page handler's method signature to